

Patch Exercise - Bug Explanations

(study notes)

Each bug below uses the four points the README asks for:
Where / How found / Root cause / Fix and why.
Line numbers refer to the original code unless marked "new".

Bug 1 - AND/OR precedence in the search query (highest priority)

- **Where:** SQL layer
 - TaskRepository.java lines 14-15 (native @Query)
 - db/queries/search_tasks.sql lines 10-13
 - db/oracle/task_search_package.sql lines 52-55 (COUNT) and 66-69 (results)
- **How found:** I read the WHERE clause and saw ... AND title LIKE ... OR description LIKE ... AND status ... with no brackets. I confirmed it with the API:
 - ?q=search returned tasks 20 and 21, which are archived.
 - ?q=api&status=OPEN returned task 2, which is IN_PROGRESS.
- **Root cause:** In SQL, AND is evaluated before OR, so the database read the condition as:
 - (archived = FALSE AND title LIKE term) OR
 - (description LIKE term AND status filter)
 - A title match ignored the status filter.
 - A description match ignored the archived filter.
 - The Oracle package and the reference query had the same bug, so the bug was copied across all three layers.

- **Fix:** Put brackets around the two LIKE checks:

```
archived = FALSE AND (title LIKE term OR
                      description LIKE term) AND (status filter)
```

- **Why:** It is the smallest change that makes the query mean what was intended. I fixed it in all three places so they stay consistent. This bug comes first because it returns wrong data: archived tasks show up and the filter can't be trusted.

Bug 2 – Artificial slowdown (Thread.sleep) in the controller

- **Where:** Backend, TaskController.java lines 35-42.
- **How found:** Reading the code, and a timing test with curl:

```
blank query      : about 1.03 s
q=a              : about 0.91 s
q=monitoring     : about 0.005 s
```

- **Root cause:** The code sleeps for $(10 - \text{query length}) \times 100$ ms. It is labelled "complexity estimation for logging" but it just blocks the request thread. Short and blank queries, including the first page load, are the slowest. It also wastes a server thread on every request.
- **Fix:** Deleted the sleep and the complexity score. I replaced the `System.out.println` with an `SLF4J log.debug`.
- **Why:** The sleep adds nothing. It also makes bug 8 visible: short queries return later than longer ones, so their results arrive out of order.

Bug 3 - Bad input crashes with HTTP 500

- Where: Backend, TaskController.java
 line 32: TaskStatus.valueOf(...)
 lines 50-53: subList(start, end)

- How found: Tried edge cases with curl:

?status=bogus	→ 500
?page=0	→ 500
?pageSize=-1	→ 500

- Root cause:

1. valueOf throws IllegalArgumentException for an unknown status, and nothing catches it.
2. page=0 gives start = -10, so subList throws IndexOutOfBoundsException.
3. pageSize had no upper limit, so a client could ask for every row at once.

- Fix:

1. Check that page ≥ 1 and that pageSize is between 1 and 100.
2. Catch the valueOf exception.

In both cases, return 400 Bad Request with a JSON {"error": "..."}, message.

- Why: A bad request is the client's fault, so it should get a 4xx with a clear message, not a 500 that looks like a server crash. The size cap protects the server. I chose an inline check over a global @ControllerAdvice to keep the change small.

Bug 4 - Pagination done in memory

- Where: Backend, TaskController.java lines 48-54 and TaskRepository.java.
- How found: Reading the code: the repository returns a List of all matching rows, then the controller calls subList to cut out one page.
- Root cause: Every request loads the full result set from the database into Java just to show 10 rows. It works for 49 rows, but memory use and response time grow with the table.
- Fix:
 1. The repository method now takes a Pageable and returns a Page<Task>.
 2. I added a countQuery so Spring runs COUNT(*) for the total.
 3. The controller passes PageRequest.of(page - 1, pageSize); Spring pages are numbered from 0, the API from 1.
 4. The SQL log confirms the database now does ... fetch first ? rows only plus the COUNT query.
- Why: The database should do the paging; that is what it is built for. The JSON response shape (items, total, page, pageSize) did not change, so the frontend needed no changes for this.

Bug 5 - LIKE wildcards in the search term are not escaped

- Where: Backend TaskController.java (building searchTerm) and the SQL in all three places.
- How found: Searched ?q=_ : it returned all 49 rows, including archived ones.
- Root cause: The user's text goes straight into the LIKE pattern. In LIKE, _ means "any one character" and % means "any characters", so typing them matches everything.
- Fix:
 1. Added escapeLike(), which puts a backslash before \, %, _.
 2. Added ESCAPE "\" to the LIKE clauses.
 3. Did the same with REPLACE in the Oracle package.
- Why: Users expect to find the text they typed, not treat special characters as wildcards. Escaping makes the search behave predictably and avoids unexpected matches.

Example:

Input: a_b%

Without escape: matches everything

With escape: matches only "a_b%"

Quick Summary

Bug	Type	Main Fix
1	SQL logic	Add brackets to AND/OR
2	Performance	Remove Thread.sleep, use SLF4J
3	Input validation	Handle exceptions, return 400, add limits
4	Pagination	Use Pageable + Page, add countQuery
5	Esarch term	Escape wildcards, use ESCAPE "\'

Key Takeaways

- Always check operator precedence (AND/OR).
- Don't block request threads (e.g., Thread.sleep).
- Validate input and return the right HTTP status (4xx vs 5xx).
- Let the database do the heavy lifting (pagination, filtering).
- Escape user input in LIKE queries.